

Seamless transition between Gaussian splatting and mesh rendering

Vid Kobal

¹Univerza v Ljubljani, Fakulteta za računalništvo in informatiko

Abstract

V seminarju predstavljamo cevovod za pripravo in prikaz hibridnih 3D objektov, sestavljenih iz mrežnega modela in Gaussian splat predstavitev. Vhodne podatke večinoma pridobimo iz fotografij, iz katerih zgradimo tako splat model kot tudi mrežni model. Oba modela nato uvozimo v Unity, ju poravnamo in med njima preklapljammo glede na razdaljo kamere. Poseben poudarek namenjamo praktični integraciji obeh predstavitev, poravnavi modelov in vizualno zveznemu prehodu med prikazoma.

1. Uvod

V 3D cevovodih lahko isti objekt predstavimo na več različnih načinov, pri čemer ima vsaka predstavitev svoje prednosti in slabosti. Klasični mrežni modeli ostajajo osrednja predstavitev v realnočasovnih pogonih, saj omogočajo učinkovito upodabljanje, standardne materiale, osvetljevanje in enostavno integracijo v obstoječe grafične cevovode. Po drugi strani pa Gaussian splatting omogoča fotorealističen prikaz in pogosto bolje ohranja drobne vizualne podrobnosti, vendar je manj primeren za geometrijske operacije.

Zaradi tega je hibridni pristop, ki združuje obe predstavitvi, smiselno predvsem kot LOD strategija za realnočasovni prikaz. V našem pristopu pri bližnjih pogledih uporabimo Gaussian splat predstavitev, ki bolje ohranja vizualne podrobnosti, pri oddaljenih pogledih pa preklapimo na geometrijsko enostavnejši mrežni model, ki omogoča cenejši prikaz. Med obema območjema izvedemo postopen prehod, s čimer skušamo ohraniti vizualno zveznost prikaza in hkrati izkoristiti prednosti obeh predstavitev.

V seminarski nalogi obravnavamo pripravo takšnih hibridnih objektov, njihovo poravnavo v Unityju ter primerjavo dveh osnovnih pristopov za mehak prehod med predstavitevama: transparentnega in dither pristopa.

Prispevek seminarja je predvsem v pripravi praktičnega cevovoda od vhodnih podatkov do hibridnega prikaza objektov v Unityju, v polavtomatski poravnavi mrežnega in splat modela, v implementaciji več načinov preklopa med obema predstavitevama ter v kvalitativni in osnovni zmogljivostni evalvaciji na več različnih objektih.

2. Sorodna dela

Gaussian splatting se je v zadnjih letih uveljavil kot učinkovit način za prikaz 3D modelov. Temeljno delo na tem področju je 3D Gaussian Splatting Kerbl idr. [KKLD23], ki uvede predstavitev scene z Gaussovimi jedri in hiter rasterizacijski postopek za interaktivni prikaz.

Ker osnovni 3D Gaussian splatting ni geometrijsko eksplicitna predstavitev, so nadaljnja dela raziskovala povezavo med splat modeli in mrežno geometrijo. SuGaR [GL24] predstavi površinsko poravnan Gaussian splatting za rekonstrukcijo mreže in mesh upodabljanje, medtem ko 2D Gaussian Splatting [HYC*24] naslavlja vprašanje geometrijske pravilnosti.

Primer dela o hibridni predstavitvi, v katerih so Gaussova jedra neposredno povezana z mrežno geometrijo je SplattingAvatar [SCR*24], ki pokaže, da je mogoče z mesh-embedded Gaussian pristopom doseči realistično realnočasovno upodabljanje, MeshGS [CLKL24] pa dodatno raziskuje poravnavo Gaussovih jeder z mrežnimi površinami.

Naše delo se osredotoča na praktičen cevovod za pripravo in uporabo hibridnih objektov v Unityju. Posebej nas zanimajo pridobivanje obeh predstavitev iz vhodnih fotografij, njuna poravnava ter vizualno zvezno preklapljanje med mrežnim in splat prikazom v realnem času.

3. Metode

3.1. Zajem podatkov

Osnovni vhod so bile večinoma fotografije realnih objektov. Isti nabor slik smo uporabili za dve rekonstrukcijski poti; za učenje Gaussian splat modela in za rekonstrukcijo mrežnega modela s fotogrametrijo.

Kadar ena od rekonstrukcijskih poti ni dala ustreznih rezultatov ali kadar smo imeli na voljo le eno model, smo manjkajoče podatke pridobili v Blenderju. Obstoječi model smo postavili v sceno, okoli njega razporedili kamere in iz njih naredili renderje, ki smo jih nato uporabili kot vhod za Gaussian Splatting ali Meshroom. Pri preprostih objektih in slabih rezultatih Meshrooma pri knjigi in prenosnem računalniku, smo mrežni model izdelali kar ročno v Blenderju.

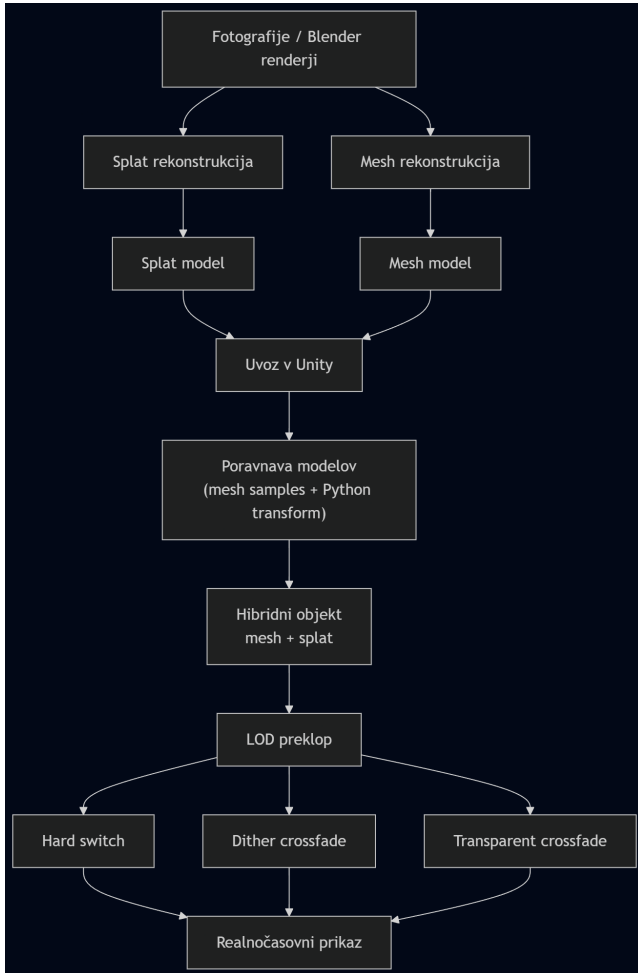


Figure 1: Pregled celotnega cevovoda.

3.2. Rekonstrukcija modelov

Gaussian splat predstavitev smo pridobili z učenjem modela iz vhodnih fotografij, pri čemer smo kot osnovni zapis uporabljali .ply. Ta predstavlja glavni vir prostorskih podatkov za splat model in referenco za nadaljnjo poravnavo ter uvoz v Unity.

Mrežni model smo rekonstruirali s fotogrametrijo v orodju Meshroom iz iste zbirke vhodnih fotografij. Pri pregostih mrežah bi geometrijo lahko dodatno poenostavili z orodjem Instant Meshes, vendar je tak postopek v našem primeru povzročil večja neskladja med mrežnim in splat modelom, zato ga v končni različici cevovoda nismo uporabili. Kadar avtomatska rekonstrukcija ni bila dovolj stabilna ali pa je bil objekt geometrijsko zelo preprost, smo mrežni model izdelali ročno.

3.3. Zajem v Blenderju

Za sintetični zajem smo v Blenderju pripravili postopek v dveh korakih. Najprej smo okoli izbranega objekta razporedili več kamer v več obhodi z različnimi višinami, nato pa smo z ločeno skripto za-

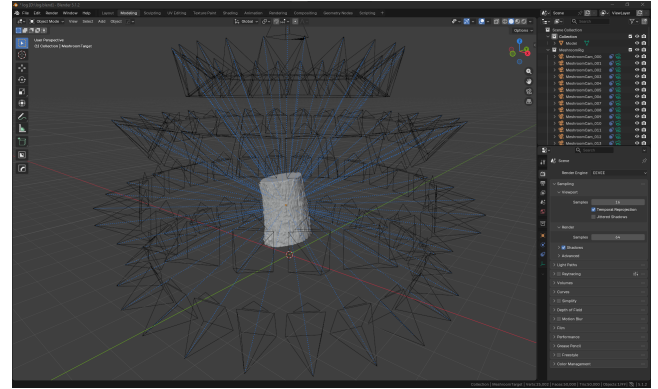


Figure 2: Postavitev virtualnih kamer okoli objekta v Blenderju za pripravo sintetičnih pogledov.

poredno izrisali poglede iz vseh kamer in tako dobili vhodne slike za Meshroom ali učenje Gaussian splat modela.

Tak pristop je uporaben takrat, ko je bilo treba rekonstruirati model iz obstoječega mesh ali splat modela ali ko realne fotografije niso bile uporabne. Hkrati smo opazili, da povsem simetrični renderji lahko povzročijo dvomnosti pri generiranju modela s fotogrametrijo.

3.4. Integracija v Unity

Obe predstavitvi smo uvozili v Unity, pri čemer smo za prikaz Gaussian splat modelov uporabili projekt UnityGaussianSplatting. Mrežni modeli so bili uvoženi kot običajni FBX. V sceni smo za vsak objekt vzpostavili hierarhijo s skupnim starševskim objektom, pod katerim sta bila ločeno pripeta mrežni in splat model.

3.5. Poravnava mrežnega in splat modela

Zato smo pripravili polavtomatski postopek poravnave, ki združuje Unity orodje za izvoz geometrije in zunanjo Python skripto za izračun transformacije. V Unityju iz mrežnega modela vzorčimo reprezentativen nabor točk po površini in ga izvozimo v JSON datoteko v lokalnem koordinatnem sistemu objekta. Python skripta nato iz izvoženega vzorca mreže in centrov Gaussianov iz datoteke .ply pripravi dve točkovni predstavitvi istega objekta.

Poravnava poteka v dveh korakih. Najprej izvedemo grobo poravnavo na podlagi osnovnih geometrijskih lastnosti obeh množic točk, nato pa transformacijo izboljšamo z iterativnim postopkom registracije točk, ki zmanjšuje odstopanje med vzorčenimi točkami mreže in centri Gaussianov. Končni rezultat je transformacija, sestavljena iz translacije, rotacije in skale, ki jo shranimo v nov JSON zapis.

V Unityju komponenta za poravnavo ta zapis prebere in ga uporabi na ciljnem objektu, s čimer dobimo dobro začetno poravnavo obeh predstavitev. Po potrebi je mogoče rezultat še ročno fino prilagoditi.

3.6. LOD preklapljanje in prehod med predstavitevama

Za preklapljanje med mesh in splat modelom smo pripravili en testni način in dva glavna pristopa za gladek prehod. Osnovni testni pristop uporablja diskretno zamenjavo med mrežnim in splat modelom glede na razdaljo kamere. Za dejanski hibridni prikaz smo nato uporabili dve različici postopnega prehoda. Prva temelji na ditheringu mrežnega modela in hkratnem fadingu splat modela, druga pa uporablja transparentni material, pri katerem se alfa vrednost mrežnega modela postopno zmanjšuje. Za izvedbo obeh prehodov je bilo treba omogočiti tudi sprotni fade vidnosti splat modela. Pri dither verziji smo uporabili lasten shader, pri transparentni verziji pa prilagojen URP/Lit shader.

Dither pristop omogoča pravilnejše prikazovanje globine objekta med samim prehodom, vendar lahko v tem območju opazimo viden šum. Transparentni pristop je vizualno mehkejši, vendar lahko povzroči značilne artefakte, pri katerih zadnji deli objekta postanejo vidni skozi sprednje dele. Za obe različici smo pripravili ločene skripte za nadzor preklopa in nastavitve mej razdalj ter obsega prehodnega območja, medtem ko so bile boljše prilagoditve shaderjev in materialov del implementacijskega dela sistema.

3.7. Podrobnosti implementacije

Pomemben del naloge je bila implementacija celotnega cevovoda v Unityju in Blenderju. Razvili smo več lastnih skript, ki povezujejo pripravo vhodnih podatkov, poravnavo in prikaz. V Blenderju smo pripravili skripte za avtomatsko postavitve kamer okoli objekta in za zaporedni izris teh kamer, izhodne slike služijo kot vhod za Gaussian Splatting ali Meshroom. V Unity smo nato pripravili orodje za vzorčenje točk po površini mrežnega modela in izvoz teh točk v JSON datoteko za zunanji postopek poravnave.

Za poravnavo smo naredili ločeno Python skripto, ki prebere izvožene vzorčne točke mreže in centre Gaussianov iz datoteke `.ply`, izračuna transformacijo med obema predstavitevama in rezultat shrani v nov JSON zapis. V Unityju smo nato razvili komponento za ponovno uporabo te transformacije v sceni ter več skript za LOD preklapljanje in nadzor prehoda med obema predstavitevama.

Za usklajen prehod med mesh in splat prikazom smo implementirali ločeno logiko za mrežni in splat del objekta. Pri splat modelu smo nad obstoječim rendererjem uvedli runtime nadzor opacit faktorja, kar omogoča postopen prikaz splatov. Pri mrežnem modelu smo implementirali dva pristopa: transparentni fade in dither fade. Transparentni pristop uporablja že obstoječ URP/Lit shader, pri katerem smo dodatno uravnavali materialne lastnosti, da bi zmanjšali premočne odboje in izboljšali barvno ujemanje s splat predstavitevjo. Za dither prehod smo pripravili lasten shader, ki uporablja diskretni vzorec za odločanje o vidnosti fragmentov glede na trenutni fade parameter. To nam omogoča, da mrežni model med prehodom ostane globinsko pravilnejši kot pri transparentnem materialu.

4. Rezultati

Pripravili smo hibridne predstavitve za več objektov, med njimi model knjige, model prenosnega računalnika in model hloda. Pri

tem je knjiga predstavljala najenostavnejši primer z najmanj kompleksnim mesh in splat modelom, medtem ko je imel model hloda največjo geometrijsko in vizualno kompleksnost. Za vse testirane objekte smo omogočili prikaz samo mrežnega modela, samo splat modela ter postopen prehod med obema predstavitevama z dither in transparentnim pristopom.

Rezultati kažejo, da je takšen hibridni prikaz praktično izvedljiv v realnem času. Vizualno najbolj prepričljive rezultate smo dosegli pri objektih, pri katerih sta se mrežni in splat model dobro ujela v geometriji in podrobnostih, medtem ko so se pri manj skladnih parih pojavila manjša odstopanja v obliki in materialnem videzu.

5. Evalvacija

Sistem smo ovrednotili z vidika zmogljivosti pri realnočasovnem prikazu in z vidika vizualne kakovosti prehoda med mesh in splat prikazom.

5.1. Eksperimentalna postavitve in metrike

Evalvacijo smo izvedli v okolju Unity na treh reprezentativnih objektih: knjigi, prenosnem računalniku in hloodu. Za vsak objekt smo primerjali prikaz samo mrežnega modela, samo splat modela ter oba mehka prehoda, dither in transparentni pristop. Meritve smo izvajali pri treh značilnih oddaljenostih kamere: bližnjem, prehodnem in oddaljenem pogledu.

Kot kvantitativne metrike smo uporabili povprečni FPS, čas sličice, čas izrisa na grafični kartici, število trikotnikov in oglišč, število izrisnih klicev ter osnovne podatke o porabi pomnilnika. Rezultate povzema tabela 1. Ker so nekatere meritve, zlasti FPS in posamezni časovni podatki, med izvajanjem opazno nihale, tabela predstavlja pregled le okvirnih povprečnih vrednosti iz opazovanih trenutkov in ne analize v daljšem časovnem intervalu. Kljub temu so rezultati dovolj jasni, da pokažejo razlike med posameznimi načini prikaza. V vseh primerih sistem deluje v realnem času, pri čemer je prikaz samo mesh modela najcenejši, medtem ko sta prikaz samo splat modela in prikaz prehoda zahtevnejša.

Tabela 1 prikazuje kvantitativni pregled vseh testiranih konfiguracij. Pri objektu knjige smo uporabili splat model z 32562 splati, pri objektu prenosnega računalnika model s 160418 splati, pri objektu hloda pa model s 243207 splati. Kvalitativna analiza je pokazala, da dither pristop praviloma bolje ohranja globinsko pravilnost, medtem ko transparentni pristop pogosto omogoča mehkejši prehod, vendar lahko povzroči artefakte prosojnosti mrežnega modela. Opaznost dither vzorca je bila odvisna predvsem od teksturne in geometrijske kompleksnosti objekta.

5.2. Vizualna primerjava

Poleg zmogljivosti smo primerjali tudi vizualno kakovost različnih pristopov. Opazovali smo predvsem vidnost dither vzorca, prosevanje zadnjih delov mreže pri transparentnem prehodu, stabilnost videza splat modela pri bližnjih pogledih in gladkost prehoda med predstavitevama.

Rezultati so pokazali, da se primernost posamezne metode razlikuje glede na objekt in stopnjo ujemanja med mrežnim in splat

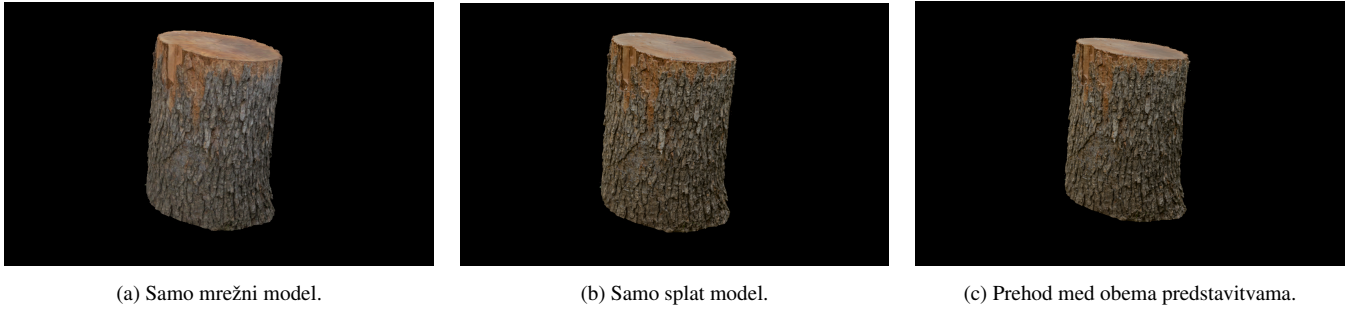


Figure 3: Primer hibridnega prikaza: mrežni model, splat model in prehod med obema predstavitevama. Ker se modela geometrijsko zelo dobro ujemata, je prehod vizualno zelo prepričljiv in v večini pogledov skoraj neopazen.

Table 1: Povzetek začetne zmogljivostne evalvacije za vse testirane objekte in načine prehoda.

Objekt	Metoda	Pogled	FPS	Čas [ms]	GPU [ms]	Tris	Oglj.	Klici
Knjiga	Dither	Near/splat	336.4	3.0	1.2	66.8k	200.4k	4
		Transition	385.0	2.6	0.8	67.7k	201.0k	9
		Far/mesh	398.5	2.5	0.2	2.6k	6.5k	7
	Transparent	Near/splat	288.1	3.5	1.7	66.8k	200.4k	4
		Transition	407.3	2.5	0.8	67.7k	201.0k	9
		Far/mesh	438.0	2.3	0.2	2.6k	6.5k	7
	Laptop	Near/splat	327.2	3.1	1.5	322.5k	967.6k	4
		Transition	366.1	2.7	0.8	326.6k	972.5k	10
		Far/mesh	419.7	2.4	0.2	5.7k	10.0k	8
	Transparent	Near/splat	245.4	4.1	1.3	322.5k	967.6k	4
		Transition	353.5	2.8	0.8	326.6k	972.5k	10
		Far/mesh	355.7	2.8	0.2	5.7k	10.0k	8
Hlod	Dither	Near/splat	346.3	2.9	1.2	488.1k	1464.3k	4
		Transition	383.2	2.6	0.9	538.1k	1491.6k	7
		Far/mesh	406.8	2.5	0.2	51.7k	32.4k	5
	Transparent	Near/splat	352.9	2.8	2.2	488.1k	1464.3k	4
		Transition	351.3	2.8	0.9	538.1k	1491.6k	7
		Far/mesh	396.9	2.5	0.2	51.7k	32.4k	5

modelom. Pri objektu knjige se je dither pristop izkazal kot globinsko pravilnejši, vendar z opaznejšim vzorcem v prehodnem območju, medtem ko je transparentni pristop ponudil mehkejši prehod, a tudi občasne globinske artefakte. Pri objektu računalnika se je dither pristop izkazal bolje, saj je bil vzorec zaradi kompleksnejše teksture skoraj neopazen, obenem pa ni povzročal težav z globino. Najbolj prepričljiv rezultat smo dobili pri objektu hloda, kjer sta se mrežni in splat model skoraj popolnoma ujemala; v tem primeru je bil transparentni prehod skoraj neopazen, dither pristop pa je hkrati bil tudi bolj pravilen kar se tiče globine mesh objekta. V obeh pristopih je mogoče vidnost značilnih artefaktov dodatno zmanjšati s krajšim prehodnim intervalom, kar sicer povzroči nekoliko ostrejši prehod, vendar zmanjša opaznost dither vzorca oziroma artefaktov prosojnosti mrežnega modela.

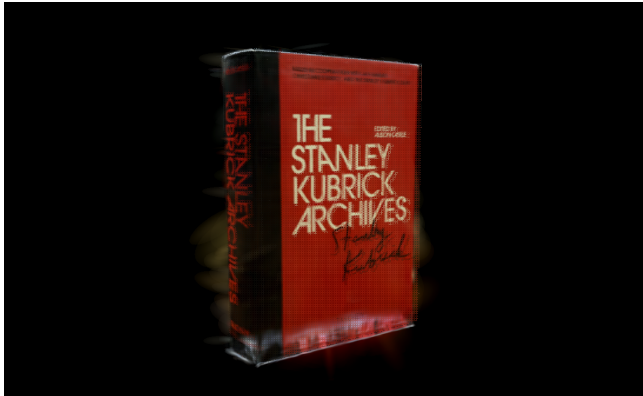
Na splošno se je pokazalo, da je transparentni pristop primernejši takrat, ko se modela zelo dobro ujemata in želimo čim mehkejši prehod, medtem ko je dither pristop varnejša izbira v primerih, kjer želimo robustnejšo globinsko pravilnost in manj transparentnih artefaktov.

6. Razprava in omejitve

Rezultati kažejo, da je hibridni pristop, ki združuje mesh in splat model, praktično uporaben za realnočasovni prikaz posameznih objektov v Unityju. Pri objektih, kjer se mrežni in splat model dobro ujemata, je mogoče doseči zelo prepričljiv in skoraj neopazen prehod, medtem ko pri manj skladnih objektih razlike v geometriji in teksturah postanejo bolj opazne.

Primerjava obeh mehkih prehodov kaže, da je najboljši pristop odvisen od samih modelov. Transparentni pristop običajno omogoča mehkejši in bolj naraven prehod, vendar je občutljiv na težave z globinsko pravilnostjo, saj lahko pri delno prosojnem mrežnem modelu postanejo zadnji deli objekta vidni skozi sprednje ploskve. Dither pristop te težave odpravi, vendar uvede viden šumni vzorec.

Pri interpretaciji rezultatov je treba upoštevati, da je bil sistem razvit in testiran na relativno majhnem številu objektov. Evalvacija zato ne meri obnašanja pri velikih scenah, temveč predvsem praktično uporabnost pri posameznih objektih. Poleg tega so rezultati



(a) Dither prehod.



(b) Transparentni prehod.

Figure 4: Primerjava obeh mehkih prehodov. Dither pristop ohranja pravilnejše globinsko obnašanje, vendar uvede viden šumni vzorec. Transparentni pristop omogoča mehkejši prehod, vendar lahko povzroči nepravilno transparentco objekta.

deloma odvisni tudi od kakovosti vhodne rekonstrukcije, zato razlike med objekti niso posledica le izbranega načina prikaza.

Dodatna omejitev sistema je, da poravnava med mrežnim in splat modelom še ni popolnoma avtomatizirana. Polavtomatski postopek običajno zagotovi dobro grobo poravnavo, vendar je pri nekaterih modelih še vedno potreben manjši ročni popravek. V prihodnjem delu bi bilo smiselno razširiti evalvacijo na večje število objektov, izboljšati avtomatsko poravnavo ter raziskati dodatne materiale oziroma shaderje.

7. Zaključek

V seminarju smo predstavili praktičen cevovod za pripravo, poravnavo in realnočasovni prikaz hibridnih 3D objektov, sestavljenih iz mrežnega modela in Gaussian splat predstavitve. Sistem zajema rekonstrukcijo obeh predstavitev, njuno integracijo v Unity, polavtomatsko poravnavo ter več načinov prehoda med njima.

Rezultati so pokazali, da sistem pri vseh testiranih objektih deluje v realnem času in da je mogoče doseči zelo prepričljive vizualne rezultate. Transparentni pristop praviloma omogoča mehkejši prehod, medtem ko dither pristop ponuja bolj pravilno globinsko obnašanje. Pokazalo se je tudi, da je kakovost končnega hibridnega prikaza močno odvisna od geometrijske in vizualne skladnosti obeh modelov.

References

- [CLKL24] CHOI J., LEE J., KIM H., LEE S.: Meshgs: Adaptive mesh-aligned gaussian splatting for high-quality rendering. In *Proceedings of the Asian Conference on Computer Vision (ACCV)* (2024). URL: https://openaccess.thecvf.com/content/ACCV2024/papers/Choi_MeshGS_Adaptive_Mesh-Aligned_Gaussian_Splatting_for_High-Quality_Rendering_ACCV_2024_paper.pdf. 1
- [GL24] GUÉDON A., LEPETIT V.: Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), pp. 5354–5363. URL: <https://openaccess.thecvf.com/content/CVPR2024/>

papers/Guedon_SuGaR_Surface-Aligned_Gaussian_Splatting_for_Efficient_3D_Mesh_Reconstruction_and_CVPR_2024_paper.pdf. 1

- [HYC*24] HUANG B., YU Z., CHEN A., GEIGER A., GAO S.: 2d gaussian splatting for geometrically accurate radiance fields. *ACM Transactions on Graphics* 43, 4 (2024). URL: <https://arxiv.org/abs/2403.17888>, doi:10.1145/3657428. 1
- [KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics* 42, 4 (2023). URL: <https://arxiv.org/abs/2308.04079>, doi:10.1145/3592433. 1
- [SCR*24] SHAO Z., CHEN Y., RAJ A., ZHANG Z., LI Y., CAO X., LIU J., ET AL.: Splattingavatar: Realistic real-time human avatars with mesh-embedded gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/papers/Shao_SplattingAvatar_Realistic_Real-Time_Human_Avatars_with_Mesh-Embedded_Gaussian_Splatting_CVPR_2024_paper.pdf. 1